Lesson 3: Writing And Running A First Java Program

Generated Jul 7, 2026 1:26 PM

What We'll Learn

By the end of this lesson, we can:

- Type and run a simple Java program.
- Explain what the Hello World program displays.
- Modify a print statement to show personalized output.

Words We'll Use

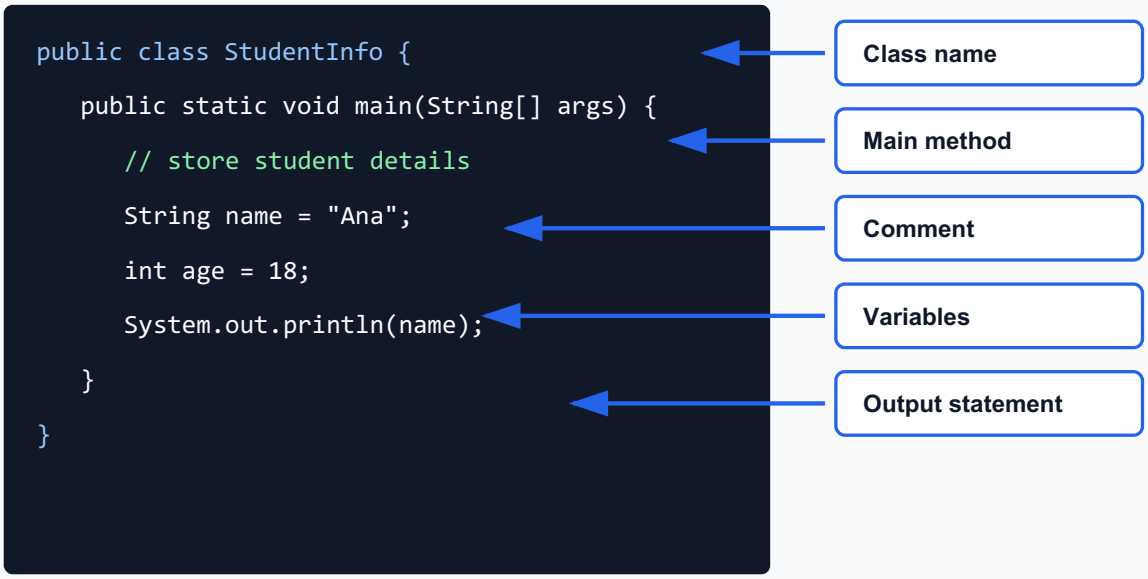
- Class - a named container for Java code.
- Main method - the starting point of a Java program.
- Statement - a single instruction.
- String - text enclosed in double quotation marks.
- Output - information displayed by a program.

Before We Start

What message would you like your first program to display, and why?

Let's Understand

Java Program Structure



Let's work through Lesson 3: Writing And Running A First Java Program together. Our focus is writing, running, and explaining a first Java output program. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means:

- Java is a programming language we use to write instructions for a computer.
- A Java program is written as source code, saved in a `.java` file, compiled, then run.
- The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode.
- An IDE helps us write, organize, run, and debug code in one workspace.
- A beginner Java program usually has a class, a main method, statements, braces, and comments.
- We read code from the outside container toward the inside instructions, then we trace what will display.

How we study it step by step:

- First, we connect the lesson to a familiar situation: What message would you like your first program to display, and why?
- Next, we name the important parts so everyone uses the same vocabulary.
- Then, we study Hello World and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Print statements and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Program output and ask: what does it do, where do we see it, and how can we check it?

- Then, we study Simple modification and ask: what does it do, where do we see it, and how can we check it?
- After that, we compare a correct example with a common wrong version.
- We trace the example slowly before running, drawing, or submitting anything.
- Finally, we explain the result in our own words so the activity is not just copying.

Rules and patterns we should remember:

- Rule: Java is case-sensitive, so `System` and `system` are not the same. Example: `System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error. Check: Which word must start with a capital S?
- Rule: Text output must be inside double quotation marks. Example: `System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen?
- Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end?
- Rule: Braces `{` and `}` must be paired because they show where a block starts and ends. Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count?
- Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting` helps us understand the next statement. Check: Will a comment display in the console?

Common mistakes we will watch for:

- Forgetting capitalization in Java keywords or class names.
- Missing quotation marks, semicolons, or closing braces.
- Thinking comments run as code.
- Changing code without predicting the output first.
- Submitting a screenshot without explaining what happened.

Mini-check before practice:

- What part of the Java program is the container?
- Where does the program start running?
- What line displays output?
- What syntax rule should we check before running?

When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Predict The Output Type: individual Time: 6 minutes Instruction: Read the Java code before running it and predict what appears. Student task:

- Underline the output statement.
- Write the exact text you think will appear.
- Run or compare with the model output.

Code / model:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, CC 102!");
    }
}
```

Output: Submit prediction, actual output, and one correction if needed.

Activity 2: Change The Message Type: hands-on coding Time: 10 minutes Instruction: Edit the printed message while keeping valid Java syntax. Student task:

- Change Hello, CC 102! to your name.
- Keep the quotation marks.
- Keep the semicolon.
- Run the program again.

Output: Submit the edited print line and output.

Activity 3: Add More Lines Type: hands-on coding Time: 10 minutes Instruction: Add more output statements to create a short profile. Student task:

- Add one line for your section.
- Add one line for your learning goal.
- Predict the order of output lines before running.

Output: Submit three output lines in the correct order.

Activity 4: Mini Debug Type: pair Time: 8 minutes Instruction: Find and fix the syntax error. Student task:

- Find the missing symbol.
- Rewrite the corrected line.
- Explain why Java needs the correction.

Code / model:

```
System.out.println("Welcome to Java!");
System.out.println("We are learning output")
```

Output: Submit corrected code and one explanation sentence.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Summarize the first Java program. Student task:

- What line displays output?
- What symbols surround text?
- What symbol ends the statement?

Output: Submit three short answers.

Now We Apply

Now we apply it through these tasks:

1. Create a program that prints your name, course, and one goal.
2. Write three output statements for a class announcement.
3. Submit a screenshot or written copy of your output.

Challenge Task

Create a Welcome Program with at least five output lines: greeting, name, course, learning goal, and motivational message. Explain each line in one sentence.

How Our Work Will Be Checked

Criteria - 20 points total

- Correctness (5): Output or answer follows the required concept and instructions.
- Completeness (4): All required parts are present.
- Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order.
- Explanation (3): Student can explain what the work does and why it is correct.
- Neatness/readability (2): Work is easy to read, label, and check.
- Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect:

- What concept from this lesson is clearest to you now?
- What mistake did you notice or correct during practice?
- Which activity helped you understand the lesson best?
- How can this skill help you design or write a Java program?
- What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning:

- Program runs without syntax errors.
- Output matches required lines.
- Student explains that println displays text.
- Completed guided-practice work with visible corrections or annotations.
- A short explanation connecting the output to the lesson target.

Student Activities

Copy and run the Hello World program. Change the output message to your name and section. Add a second output line with your favorite subject. Predict the output before running the program. Compare print and println behavior. Mini Debug: fix a missing quotation mark. Create a program that prints your name, course, and one goal. Write three output statements for a class announcement. Submit a screenshot or written copy of your output.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.